

РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ ДЛЯ ФИТНЕСА НА ПЛАТФОРМЕ ANDROID

А.Р.Вафин

Воронежский государственный университет

Введение

Мобильные фитнес-приложения стали не просто инструментами отслеживания, а полноценными цифровыми тренерами. Разработка таких приложений для Android особенно актуальна благодаря доминированию платформы на рынке и глобальному тренду на здоровый образ жизни. Современные технологии позволяют создавать интуитивные интерфейсы с обширными каталогами тренировок, доступными как новичкам, так и профессионалам. Таким образом, создание фитнес-приложения для Android решает комплексную задачу на стыке разработки ПО, спортивной медицины и психологии, внося вклад в улучшение качества жизни населения [1, 2].

Цель работы заключается в создании Android-приложения для занятий фитнесом, обеспечивающего доступ к систематизированной базе тренировок и функциональность для формирования персонализированных программ.

Для достижения поставленной цели требуется решить следующие задачи:

- спроектировать и разработать пользовательский интерфейс, обеспечивающий удобную навигацию, визуализацию каталога упражнений и возможность конструирования индивидуальных планов;
- реализовать модуль хранения и обработки данных, включая проектирование локальной базы и механизмы синхронизации с удаленным сервером;
- разработать бизнес-логику приложения, охватывающую сценарии проведения тренировок, фиксацию прогресса пользователя и анализ показателей сна;
- создать серверную часть (бэкенд), включая разработку API для аутентификации пользователей и облачного хранения данных;
- провести всестороннее тестирование клиентской и серверной частей для проверки корректности работы интерфейса и алгоритмов [3].

1. Реализация программного продукта

В качестве языка программирования выбран Kotlin благодаря его современности, лаконичности и таким преимуществам, как null-безопасность, корутины, data-классы и расширения. Полная совместимость с Java позволяет использовать существующие библиотеки и обеспечивает плавную миграцию. Средой разработки служит Android Studio, предоставляющая полный цикл создания приложений. Внутри неё используется Gradle для управления зависимостями и сборкой проекта. Для контроля версий применяется Git, обеспечивающий отслеживание изменений и совместную работу над кодом. Для создания пользовательского интерфейса выбран Jetpack Compose — современный декларативный фреймворк, позволяющий описывать UI непосредственно в коде Kotlin. Это упрощает разработку динамических интерфейсов с поддержкой анимаций и Material Design 3, сокращая объем шаблонного кода [4].

Работу с базой данных обеспечивает Room — высокоуровневая библиотека над SQLite, автоматизирующая создание таблиц, выполнение запросов и миграции. Интеграция с LiveData и Flow упрощает обработку данных в реальном времени. Для сетевого взаимодействия с REST API используется Retrofit — популярная библиотека, позволяющая описывать запросы через интерфейсы и аннотации. В связке с OkHttp она обеспечивает высокую производительность, автоматическую сериализацию данных и обработку ошибок. Тестирование серверной части выполняется с помощью Postman — инструмента для отправки HTTP-запросов различных типов, настройки заголовков и автоматизации проверок через коллекции [5].

В основе архитектуры лежит набор принципов Clean Architecture (рис. 1), которые подразумевают разделение приложения на три слоя: доменный слой, в котором находятся все контракты приложения, сценарии использования и модели данных, слой интерфейса и слой данных, отвечающий за взаимодействие с компонентами системы в рамках бизнес-логики. Использование данного подхода позволяет менять разные части приложения изолированно, что положительно сказывается на процессе разработки [6].

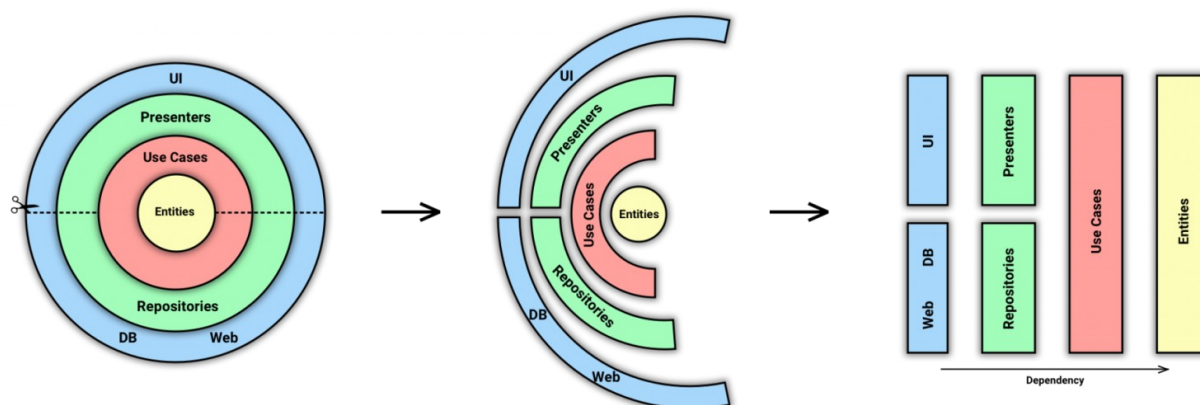


Рис 1. Архитектурные слои приложения

Для разделения представления и логики управления данными в приложении используется паттерн MVVM (Model-View-ViewModel) (рис. 2). Этот паттерн обеспечивает чёткое разделение обязанностей между компонентами приложения:

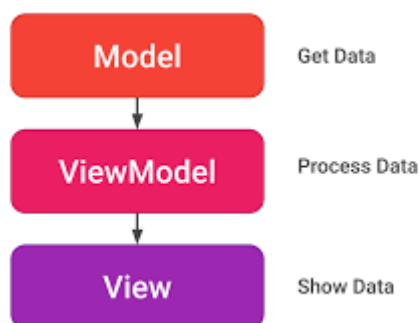


Рис 2. Паттерн проектирования MVVM

Для разделения логики и интерфейса в приложении используется паттерн MVVM (Model-View-ViewModel), который четко распределяет обязанности между компонентами:

- Model (Модель): отвечает за бизнес-логику и данные. Она взаимодействует с источниками данных (например, базой данных или сервером) и передает полученную информацию во ViewModel.

- View (Представление): это пользовательский интерфейс. Его задача — отображать данные и реагировать на действия пользователя. View подписано на изменения во ViewModel и автоматически обновляется при поступлении новых данных.
- ViewModel (Модель представления): выступает связующим звеном. Она получает данные от Model, подготавливает их для отображения во View, а также обрабатывает команды пользователя, поступающие из интерфейса, и при необходимости обновляет Model.

В рамках работы также была разработана схема SQLite базы данных приложения (рис. 3), в которой хранится информация о упражнениях и тренировках, истории тренировок а также истории сна [7].

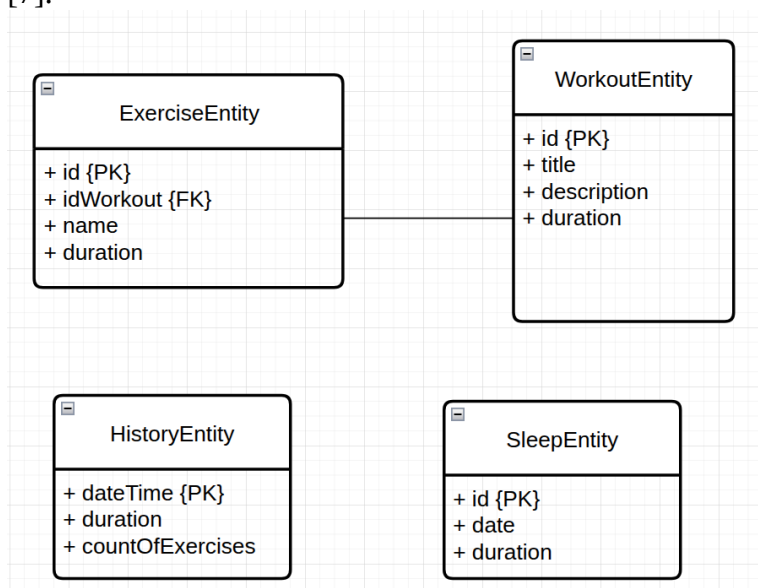


Рис 3. Схема SQLite базы данных приложения

2. Пользовательский интерфейс

Для создания пользовательского интерфейса используется Jetpack Compose — современный декларативный фреймворк от Google. Он позволяет описывать интерфейс непосредственно в коде Kotlin, что ускоряет разработку и сокращает объем шаблонного кода. Благодаря декларативному подходу упрощается создание динамичных экранов, а встроенная поддержка анимаций и Material Design 3 помогает реализовать современный дизайн [8]. Экран регистрации и входа (рис. 4, рис. 5) служит изначальной точкой входа в приложение до авторизации. На главном экране (рис. 6) показана библиотека тренировок с возможностью добавить свою собственную программу. Пользователь может открыть любую тренировку и выполнить ее, а также открыть историю тренировок и настройки.

На экране истории (рис. 7) пользователь может посмотреть историю своих тренировок, их длительность и количество упражнений.

На экране трекинга сна (рис. 8) пользователь может посмотреть длительность, дату и время сна, а также добавить или удалить записи о сне. Более того, этот экран, с использованием интеллектуального модуля советов и подсчета средней продолжительности сна определяет, достаточно ли для тренировок спит пользователь, и, если нет, отображает предупреждение о возможных последствиях.

Экран «Профиль» (рис. 9) позволяет пользователю увидеть информацию о нём, сообщить об ошибке и зайти в трекинг сна [9].

Конференция школьников, студентов и молодых ученых

Последний экран, экран тренировки (рис. 10, рис. 11, рис. 12), показывает пользователю детальную информацию о составе тренировки с возможностью выбора определенных упражнений, а также таймер с музыкой и отдыхом после каждого упражнения с возможностью приостановить тренировку или закончить без записи в историю.

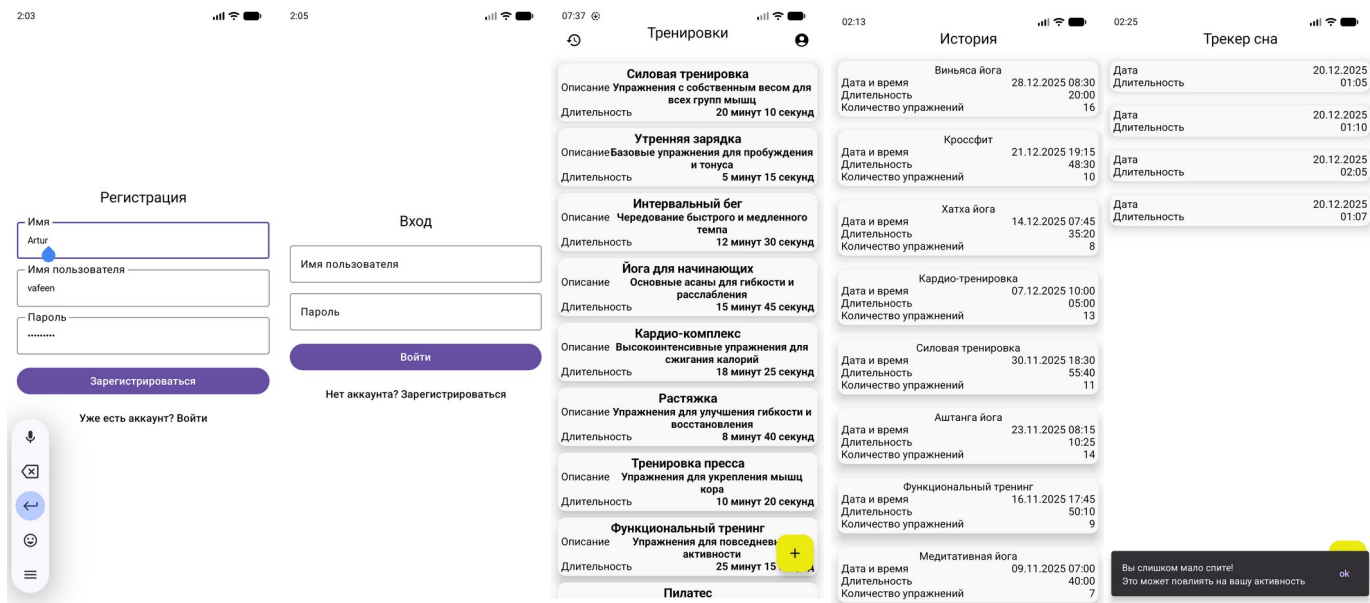


Рис 4. Регистрация Рис 5. Вход Рис 6. Главный экран Рис 7. История Рис 8. Трекинг сна

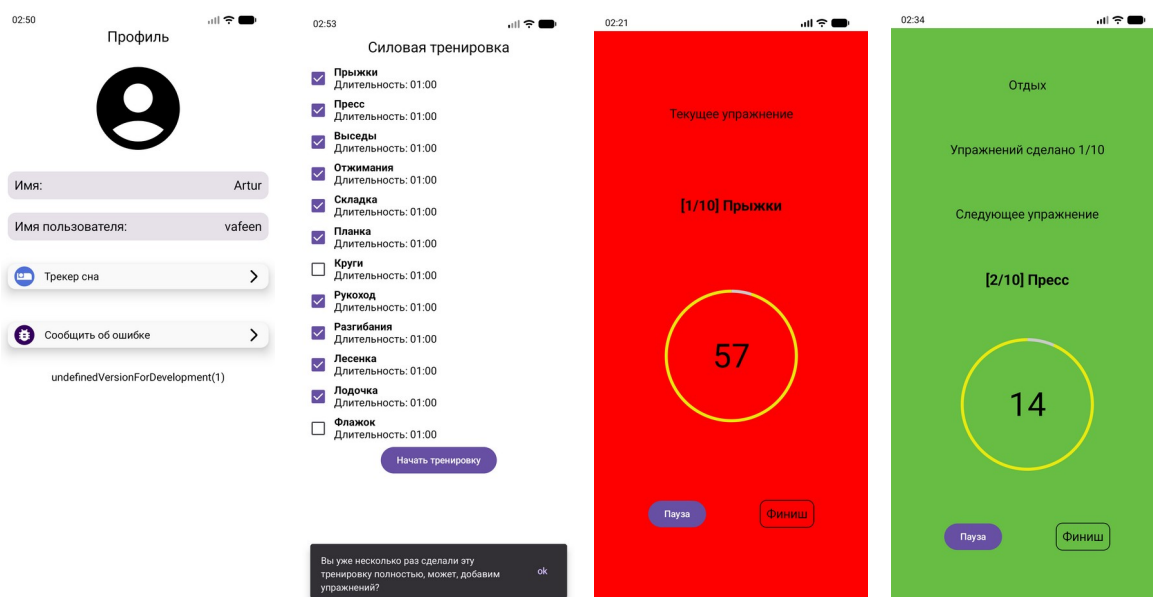


Рис 9. Профиль Рис 10. Упражнения Рис 11. Тренировка Рис 12. Отдых

Заключение

В ходе выполнения работы было создано мобильное фитнес-приложение для устройств на платформе Android. Оно предлагает комплексный подход к организации

тренировок и контролю за показателями здоровья. Разработанное решение вобрало в себя лучшие черты существующих аналогов, дополнив их новыми возможностями, которые делают приложение более гибким, адаптивным и ориентированным на индивидуальные потребности пользователя [10].

Основные результаты работы можно представить следующим образом:

- реализован полный цикл разработки: создана клиентская часть с понятным интерфейсом и серверное приложение, обеспечивающее безопасную авторизацию и хранение данных через защищенное API;
- обеспечена автономность и синхронизация: разработан модуль работы с данными, включающий локальную базу для стабильной работы без интернета и надежный механизм синхронизации с облачным сервером;
- уделено внимание безопасности: для защиты личной информации пользователей (логинов, данных о прогрессе, персональных программ) используется защищенное соединение HTTPS с SSL-сертификатами;
- реализована ключевая логика приложения: создана основа, управляющая процессом тренировок, сбором статистики и отслеживанием показателей восстановления организма;
- проведено всестороннее тестирование: выполнена комплексная проверка, включающая оценку удобства интерфейса, корректности работы клиентской и серверной частей, а также точности основных алгоритмов.

Все задачи, поставленные в начале работы, были полностью решены. Интерфейс спроектирован и реализован с помощью Jetpack Compose, локальное хранение данных организовано через Room, создана бизнес-логика для фильтрации тренировок и отслеживания прогресса, а итоговая работоспособность системы подтверждена тестированием.

Приложение имеет хороший потенциал для дальнейшего развития. В будущем его можно дополнить такими функциями, как соревновательные челленджи, интеграция с умными часами и фитнес-браслетами для автоматического сбора данных о сне и активности, а также функция автоматического определения начала тренировки на основе сигналов с носимых устройств [11].

Таким образом, разработанное решение не только восполняет недостаток гибкости, присущий многим существующим фитнес-приложениям, но и закладывает фундамент для создания интеллектуальной системы персонального сопровождения. Такая система способна подстраиваться под индивидуальные особенности и актуальное состояние каждого человека, делая тренировки более эффективными и безопасными [12].

Работа выполнена под руководством кандидата физ.-мат. наук, доцента кафедры кибербезопасности информационных систем Воронежского государственного университета Болотовой Светланы Юрьевны.

Литература

1. Жемеров, Д. Kotlin в действии / Д. Жемеров, С. Исакова. – Москва : ДМК Пресс, 2018. – 402 с.
2. Колесниченко, И. Программирование под Android / И. Колесниченко. – Санкт-Петербург : Питер, 2019. – 912 с.
3. Официальная документация Google по Android [Электронный ресурс]. – Режим доступа: <https://developer.android.com> (дата обращения: 2026).
4. Мартин, Р. Чистая архитектура / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с.

5. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург : Питер, 2021. – 496 с.
6. Фаулер, М. Рефакторинг. Улучшение проекта существующего кода / М. Фаулер. – Санкт-Петербург : Питер, 2020. – 448 с.
7. Официальная документация Kotlin [Электронный ресурс]. – Режим доступа: <https://kotlinlang.org/docs/home.html> (дата обращения: 2026).
8. Макконнелл, С. Совершенный код / С. Макконнелл. – Москва : Русская редакция, 2019. – 896 с.
9. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – Москва : Вильямс, 2016. – 816 с.
10. Зайцев, В. Наука и практика силовых тренировок / В. Зайцев. – Москва : Олимп-Бизнес, 2022. – 320 с.
11. Дельватор, Ф. Анатомия силовых упражнений для мужчин и женщин / Ф. Дельватор. – Минск : Попурри, 2021. – 224 с.
12. Брукс, Ф. Мифический человеко-месяц, или Как создаются программные системы / Ф. Брукс. – Москва : Символ-Плюс, 2020. – 304 с.

Вафин Артур Русланович – студент 4-го курса кафедры Математического Обеспечения ЭВМ Воронежского государственного университета. E-mail: 666av6@gmail.com